

Stack

(Stack, Arithmetic Expression)

What is a stack?

- Stores a set of elements in a particular order
- Stack principle: **LAST IN FIRST OUT**
- = **LIFO**
- It means: the last element inserted is the first one to be removed
- Example



- Which is the first element to pick up?

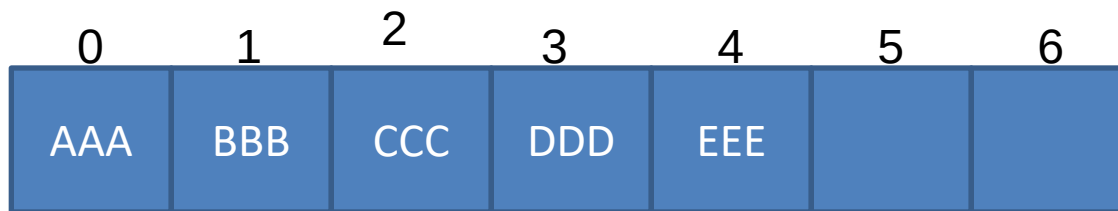
- Applications of Stack in Real Life
- Stack is used very often in real life, even normal people use applications of Stack in their daily life routines. Here is some example of the stack in real-life. Below are the 10 Applications of Stack in Real Life.
- Women's Bangles: Women wear a bangle one by one and to pull the first one they have to first pull out the last one.
- Books and Clothes: Piled on top of each other is a great example of the stack.
- Floors in a Building: A person is living on the top floor and wants to go outside, he/she first needs to land on the ground floor.
- Browsers: Web browsers use the stack to keep track of the history of websites if you click back then the previous site opens immediately.
- Mobile Phone Call Logs: Call logs in mobiles to use the stack, to get a first-person call log you have to scroll.
- Companies: When a company want to reduce its workforce. Typically they follow "last hired, first fired"
- Garage: If a garage is not wide enough. To remove the first car we have to take out all the other cars after it.
- Tubewell Boring Machine: Tubewell boring machines use a stack to pull or push same as the stack
- Text Editors: Undo or Redo mechanism in the Text Editors(Excel, Notepad or WordPad etc.)
- The CD/DVD stand

- Application of Stack in Data Structure
- Apart from the real-life example of stack here are some different applications of the stack in Data Structure. The functionality will be the same else we can't say it stacks.
- Memory management
- Function Call(recursive functions.)
- String Reversal
- Parenthesis Checking
- Backtracking
- Syntax Parsing
- Reversing a String
- Matching HTML Tags in Web Developing.
- Arithmetic Expression Evaluation
- Java compiler uses postfix notation
- Java virtual machine uses a stack
- Expression Conversion or Expression Evaluation

Stack

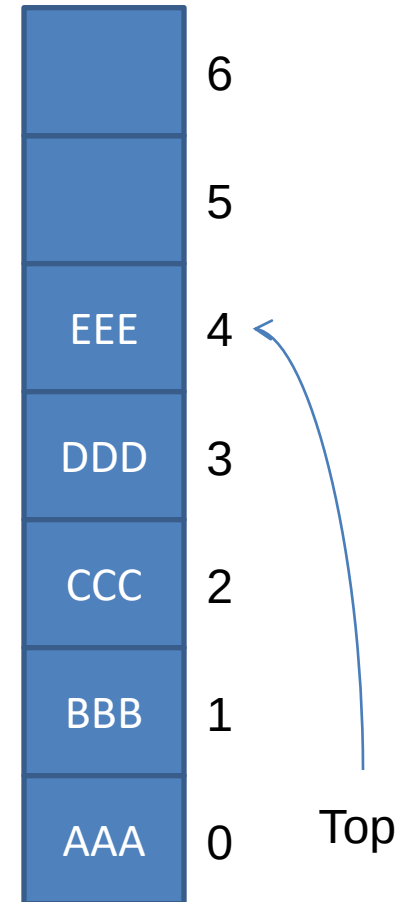
- It is a linear data structure consisting of list of items.
- In stack, data elements are added or removed only at one end, called ***the top of the stack***.
- Two basic operations are associated with stack:
 1. “Push” operation is used to insert an element into a stack.
 2. “Pop” operation is used to delete an element from a stack.
- Example:

Stack: AAA, BBB, CCC, DDD, EEE



Top

Figure: Diagrams of Stacks



Algorithms for Push and Pop Operations

For Push Operation

Push-Stack(Stack, Top, MaxSTK, Item)

Here, Stack is the place where to store data. Top represents in which location the data is to be inserted. MaxSTK is the maximum size of the stack and finally, Item is the new item to be added.

- 1.If $Top = MaxSTK$ then Print: Overflow and Return. /*...Stack already filled..*/
- 2.Set $Top := Top + 1$
- 3.Set $Stack[Top] := Item$
- 4.Return.

For Pop Operation

Pop-Stack(Stack, Top, Item)

Here, Stack is the place where data are stored. Top represents from which location the data is to be removed. Top element is assigned to Item.

- 1.If $Top = -1$ then Print: Underflow and Return. */*...Stack already Empty..*/*
- 2.Set $Item := Stack[Top]$
- 3.Set $Top := Top - 1$
4. Return.

Push & Pop Example

Consider the following stack of characters, where STACK is allocated $N = 8$ memory cells:

STACK: A, C, D, F, K, __, __, __

(For notational convenience, we use “__” to denote an empty memory cell.) Describe the stack as the following operations take place:

- | | |
|----------------------|----------------------|
| (a) POP(STACK, ITEM) | (e) POP(STACK, ITEM) |
| (b) POP(STACK, ITEM) | (f) PUSH(STACK, R) |
| (c) PUSH(STACK, L) | (g) PUSH(STACK, S) |
| (d) PUSH(STACK, P) | (h) POP(STACK, ITEM) |

The POP procedure always deletes the top element from the stack, and the PUSH procedure always adds the new element to the top of the stack. Accordingly:

- (a) STACK: A, C, D, F, __, __, __, __
(b) STACK: A, C, D, __, __, __, __, __
(c) STACK: A, C, D, L, __, __, __, __
(d) STACK: A, C, D, L, P, __, __, __
(e) STACK: A, C, D, L, __, __, __, __
(f) STACK: A, C, D, L, R, __, __, __
(g) STACK: A, C, D, L, R, S, __, __
(h) STACK: A, C, D, L, R, __, __, __

Application of Stack

- **Arithmetic Expression Evaluation**

Calculators use a stack structure to hold values for calculation.

- **Syntax Parsing**

Many compilers use a stack for parsing the syntax of expressions before translating into low level code.

- **Solving Search Problem**

Solving a search problems, regardless of whether the approach is exhaustive or optimal, needs stack space. Example of exhaustive search methods is backtracking. Example of optimal search exploring methods is branch and bound. All of these algorithms use stacks to remember the search nodes that have been noticed but not explored yet.

- **Runtime Memory Management**

Almost all computer runtime memory environments use a special stack (the “call stack”) to hold information about procedure/function calling and nesting in order to switch to the context of the called function and restore to the caller function when the calling finishes. They follow a runtime protocol between caller and callee to save arguments and return value on the stack. Stacks are an important way of supporting nested or recursive function calls.

Arithmetic Expression

- Arithmetic expression consists of operands and operators.
- Three types of arithmetic expression:
 1. Infix Expression
 - Operators are placed between its two operands.
 - Example: $2 + 4$, $a - b / c$
 2. Prefix Expression(Polish Notation)
 - Operators are placed before its two operands.
 - Example: $+ 2 4$, $- a / b c$
 3. Postfix Expression (Reverse Polish Notation or RPN)
 - Operators are placed after its two operands.
 - Example: $2 4 +$, $a b c / -$

Postfix Expression

From Postfix to Answer

Ex: 10 2 8 * + 3 -

- First, push(10) into the stack

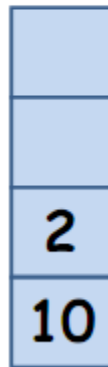


Postfix..

From Postfix to Answer

Ex: 10 2 8 * + 3 -

- Then, push(2) into the stack



Postfix..

From Postfix to Answer

Ex: 10 2 8 * + 3 -

- Push(8) into the stack

8
2
10

Postfix..

From Postfix to Answer

Ex: 10 2 8 * + 3 -

8
2
10

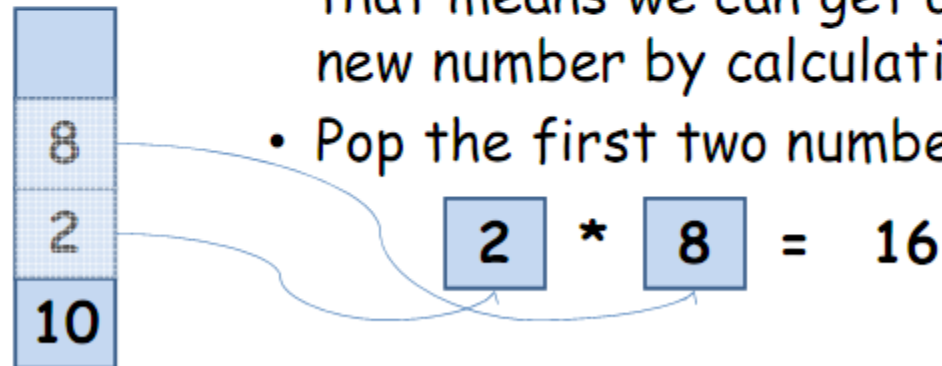
- Now we see an operator *, that means we can get a new number by calculation

Postfix..

From Postfix to Answer

Ex: 10 2 8 * + 3 -

- Now we see an operator *, that means we can get a new number by calculation
- Pop the first two numbers

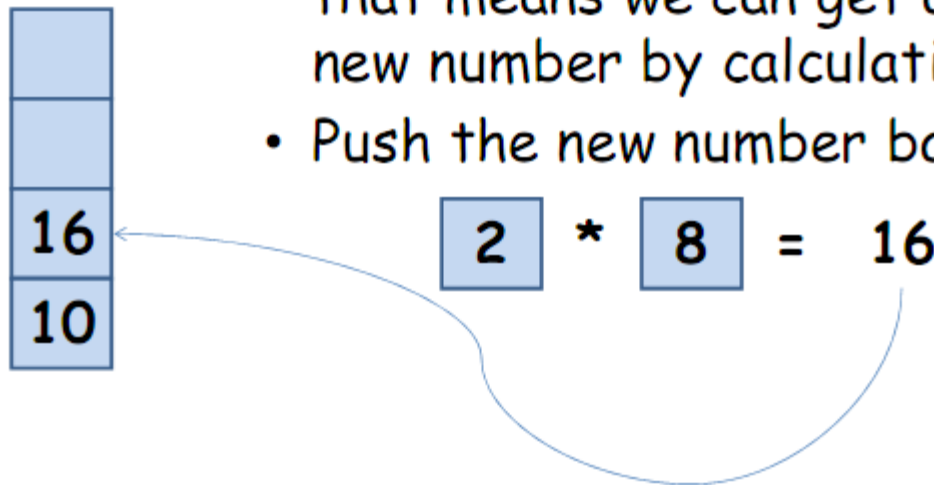


Postfix..

From Postfix to Answer

Ex: 10 2 8 * + 3 -

- Now we see an operator *, that means we can get a new number by calculation
- Push the new number back

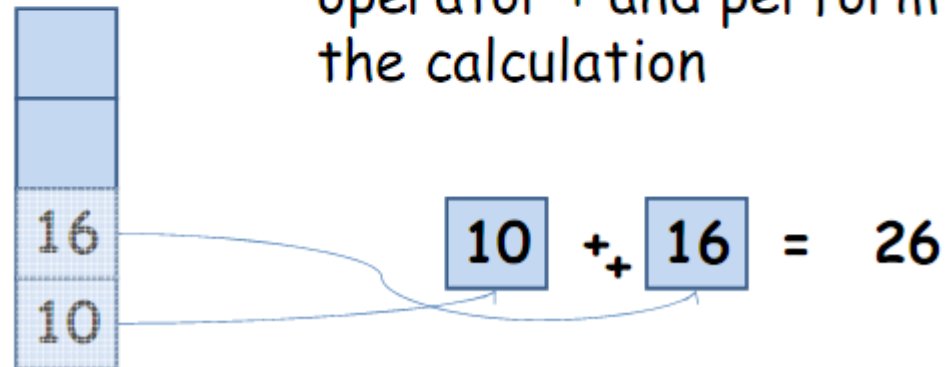


Postfix..

From Postfix to Answer

Ex: 10 2 8 * + 3 -

- Then we see the next operator + and perform the calculation



Postfix..

From Postfix to Answer

Ex: 10 2 8 * + 3 -



- Then we see the next operator + and perform the calculation
- Push the new number back

$$\boxed{10} + \boxed{16} = 26$$

Postfix..

From Postfix to Answer

Ex: 10 2 8 * + 3 -

- We see the next number 3
- Push (3) into the stack



Postfix..

Compute the Answer

Ex: 10 2 8 * + 3 -

- The last operation



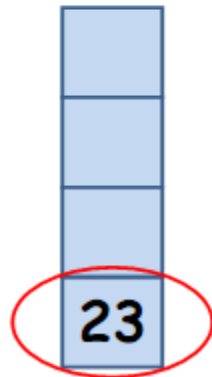
$$\boxed{26} - \boxed{3} = 23$$

Postfix..

From Postfix to Answer

Ex: 10 2 8 * + 3 -

- The last operation



$$\boxed{26} - \boxed{3} = \boxed{23}$$

answer!

Consider the following arithmetic expression P written in postfix notation:

P: 5, 6, 2, +, *, 12, 4, /, -

(Commas are used to separate the elements of P so that 5, 6, 2 is not interpreted as the number 562.) The equivalent infix expression Q follows:

Q: $5 * (6 + 2) - 12 / 4$

Symbol Scanned		STACK
(1)	5	5 ✓
(2)	6	5, 6
(3)	2	5, 6, 2
(4)	+	5, 8 ✓
(5)	*	40 ✓
(6)	12	40, 12
(7)	4	40, 12, 4
(8)	/	40, 3
(9)	-	37
(10)	)	

Consider the postfix expression P in Problem 6.8. Evaluate P using Algorithm

First add a sentinel right parenthesis at the end of P to obtain:

P: 12, 7, 3, -, /, 2, 1, 5, +, *, +,)

Symbol	STACK
12	12
7	12, 7
3	12, 7, 3
-	12, 4
/	3
2	3, 2
1	3, 2, 1
5	3, 2, 1, 5
+	3, 2, 6
*	3, 12
+	15
)	15

$P_1: 5, 3, +, 2, *, 6, 9, 7, -, /, -$
 $P_2: 3, 5, +, 6, 4, -, *, 4, 1, -, 2, \uparrow, +$
 $P_3: 3, 1, +, 2, \uparrow, 7, 4, -, 2, *, +, 5, -$

Consider the above postfix expressions. Evaluate the value the postfix expression by using stack.

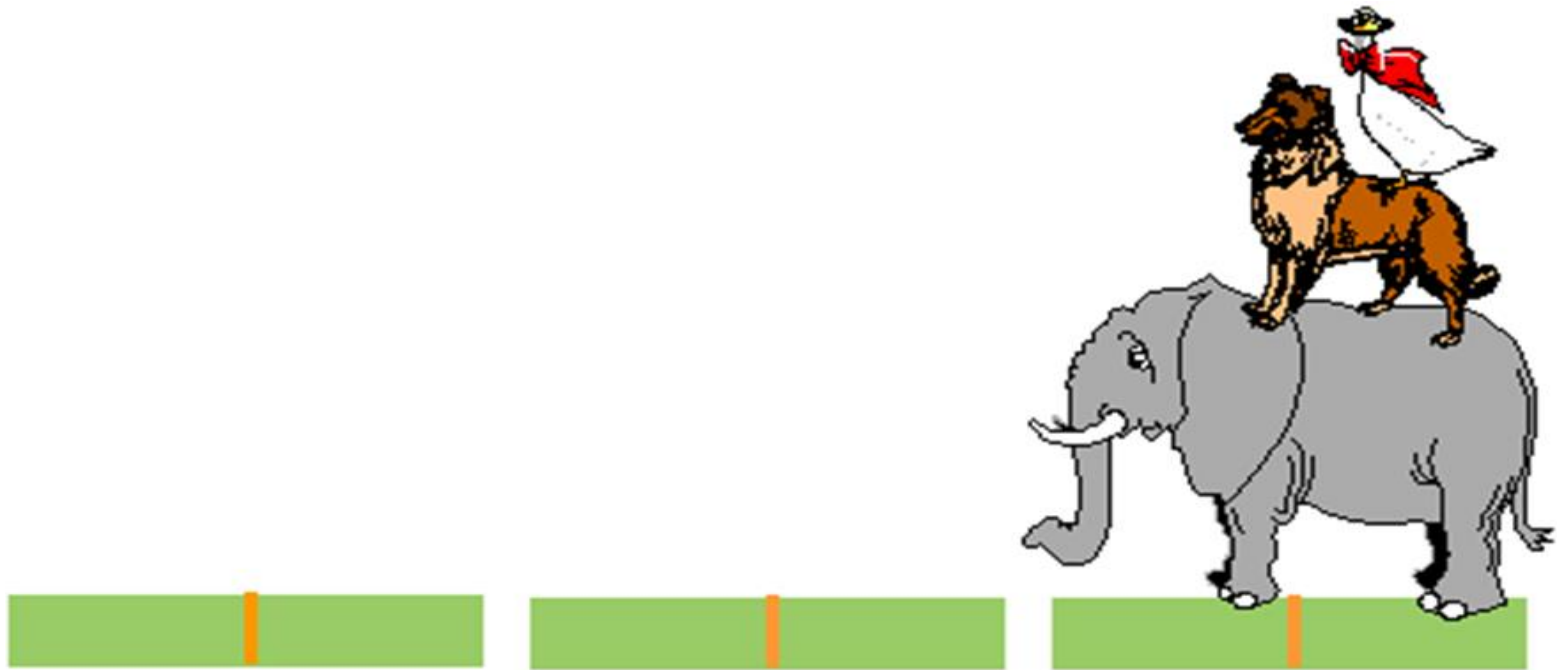
The Towers of Hanoi

A Stack-based Application

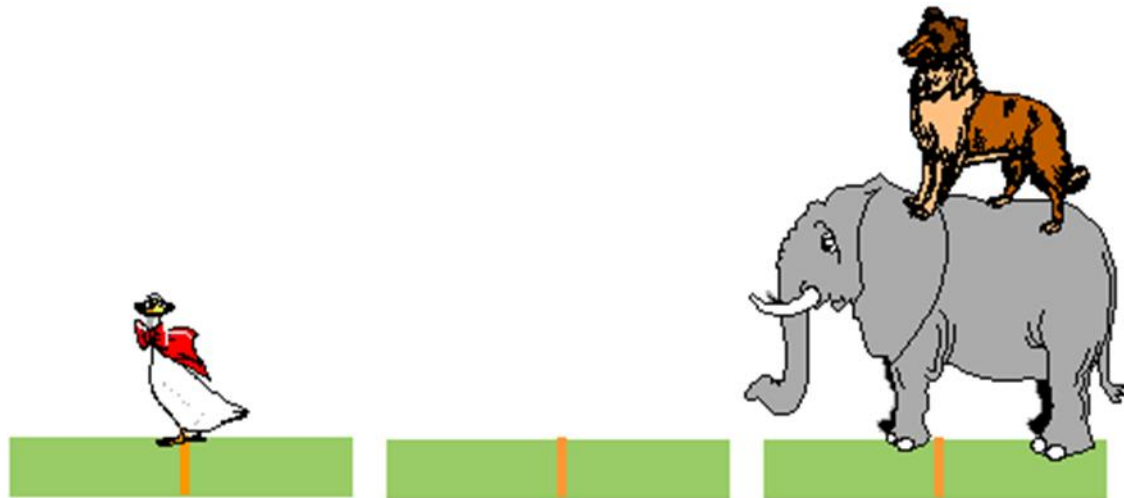
- ❖ GIVEN: three poles
- ❖ a set of discs on the first pole, discs of different sizes, the smallest discs at the top
- ❖ GOAL: move all the discs from the left pole to the right one.
- ❖ CONDITIONS: only one disc may be moved at a time.
- ❖ A disc can be placed either on an empty pole or on top of a larger disc.



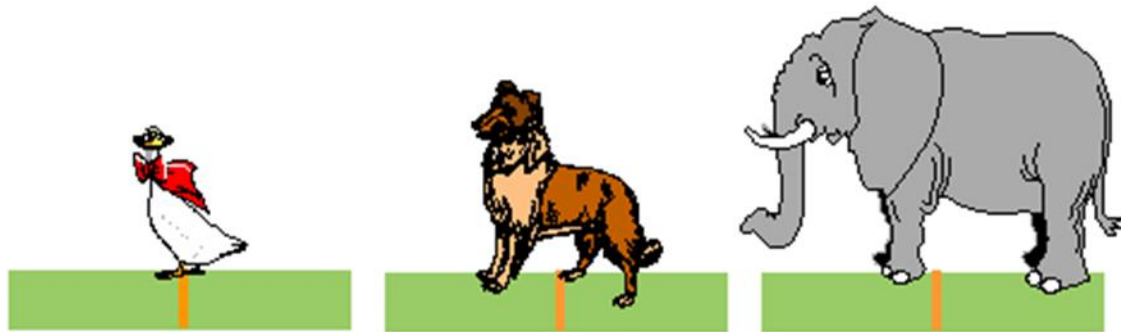
Towers of Hanoi



Towers of Hanoi



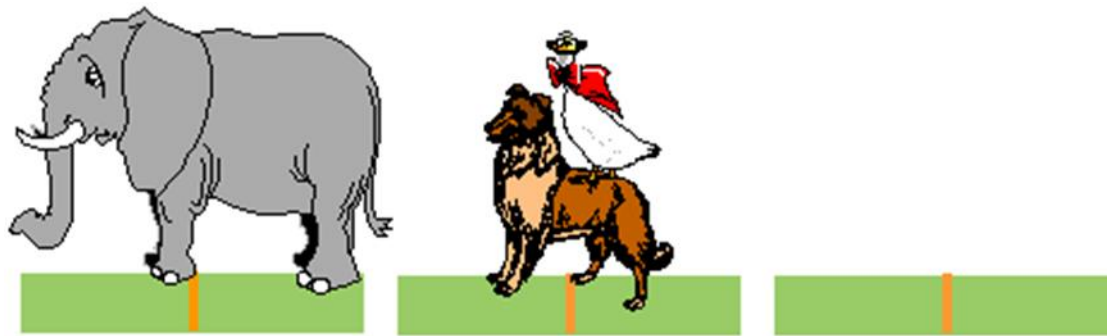
Towers of Hanoi



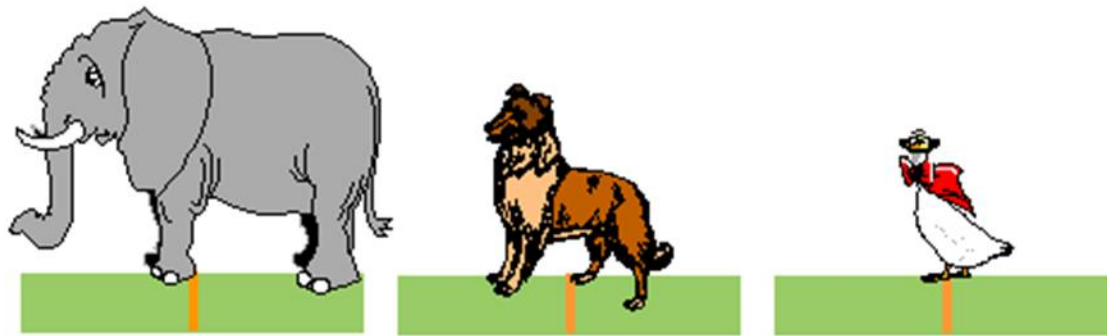
Towers of Hanoi



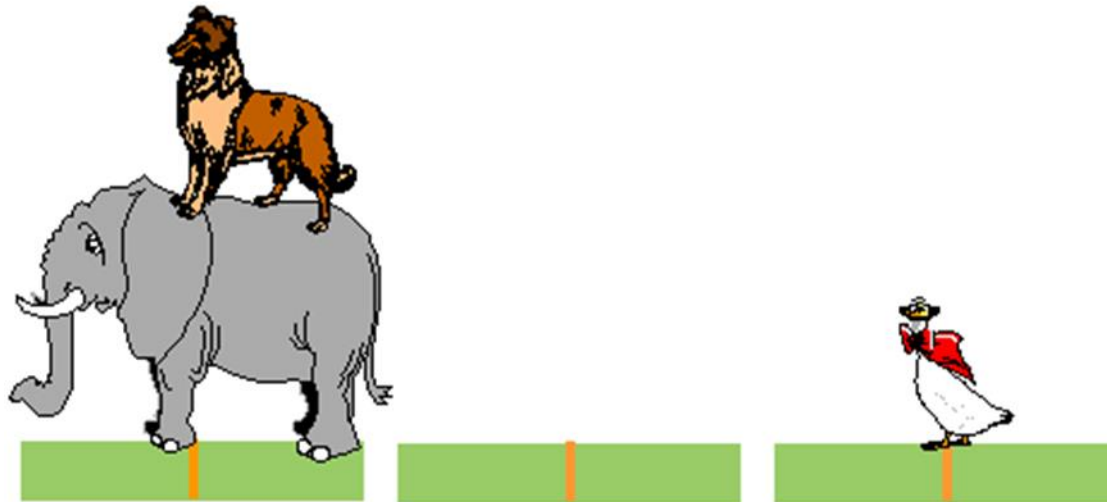
Towers of Hanoi



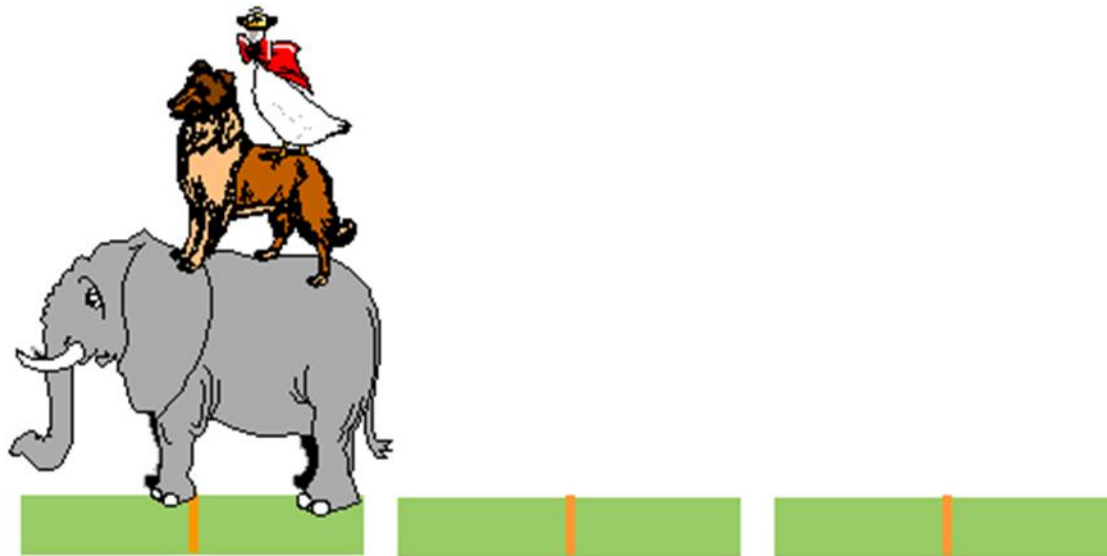
Towers of Hanoi



Towers of Hanoi



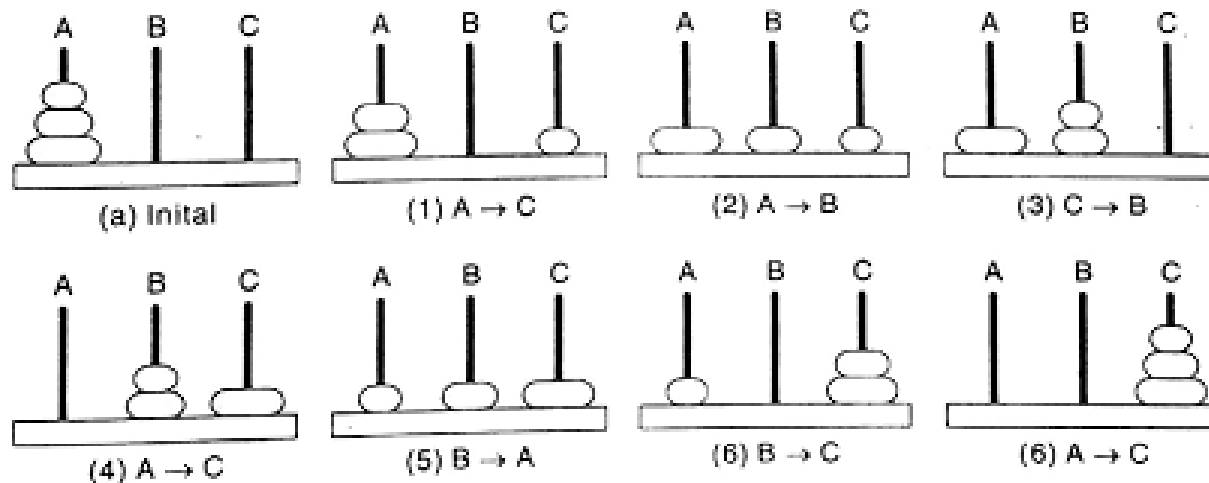
Towers of Hanoi



Towers of Hanoi

The solution to the Towers of Hanoi problem for $n = 3$ appears in Fig. 6.15. Observe that it consists of the following seven moves:

- $n = 3$:
- Move top disk from peg A to peg C.
 - Move top disk from peg A to peg B.
 - Move top disk from peg C to peg B.
 - Move top disk from peg A to peg C.
 - Move top disk from peg B to peg A.
 - Move top disk from peg B to peg C.
 - Move top disk from peg A to peg C.



Towers of Hanoi

In other words,

$n = 3$: $A \rightarrow C$, $A \rightarrow B$, $C \rightarrow B$, $A \rightarrow C$, $B \rightarrow A$, $B \rightarrow C$, $A \rightarrow C$

For completeness, we also give the solution to the Towers of Hanoi problem for $n = 1$ and $n = 2$:

$n = 1$: $A \rightarrow C$

$n = 2$: $A \rightarrow B$, $A \rightarrow C$, $B \rightarrow C$

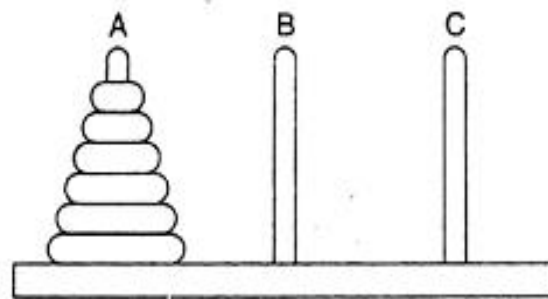
Note that $n = 1$ uses only one move and that $n = 2$ uses three moves.

Rather than finding a separate solution for each n , we use the technique of recursion to develop a general solution. First we observe that the solution to the Towers of Hanoi problem for $n > 1$ disks may be reduced to the following subproblems:

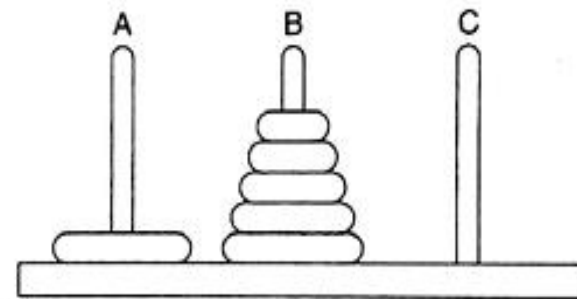
- (1) Move the top $n - 1$ disks from peg A to peg B.
- (2) Move the top disk from peg A to peg C: $A \rightarrow C$.
- (3) Move the top $n - 1$ disks from peg B to peg C.

Towers of Hanoi

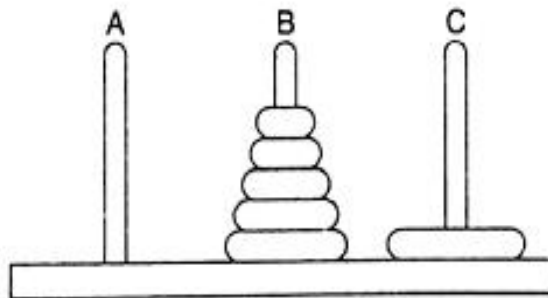
This reduction is illustrated in Fig. 6.16 for $n = 6$. That is, first we move the top five disks from peg A to peg B, then we move the large disk from peg A to peg C, and then we move the top five disks from peg B to peg C.



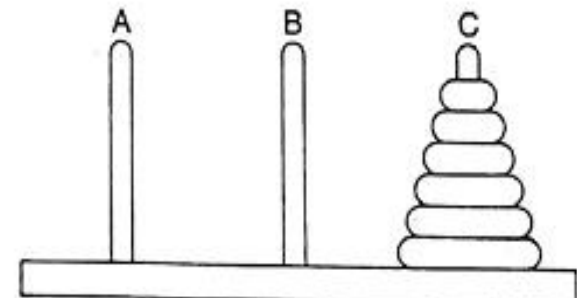
(a) Initial: $n = 6$



(b) Move top five disks from peg A to peg B



(c) Move top disk from peg A to peg C



(d) Move top five disks from peg B to peg C

Towers of Hanoi

Let us now introduce the general notation

$\text{TOWER}(N, \text{BEG}, \text{AUX}, \text{END})$

to denote a procedure which moves the top n disks from the initial peg BEG to the final peg END using the peg AUX as an auxiliary. When $n = 1$, we have the following obvious solution:

$\text{TOWER}(1, \text{BEG}, \text{AUX}, \text{END})$ consists of the single instruction $\text{BEG} \rightarrow \text{END}$

Furthermore, as discussed above, when $n > 1$, the solution may be reduced to the solution of the following three subproblems:

- (1) $\text{TOWER}(N - 1, \text{BEG}, \text{END}, \text{AUX})$
- (2) $\text{TOWER}(1, \text{BEG}, \text{AUX}, \text{END})$ or $\text{BEG} \rightarrow \text{END}$
- (3) $\text{TOWER}(N - 1, \text{AUX}, \text{BEG}, \text{END})$

Towers of Hanoi

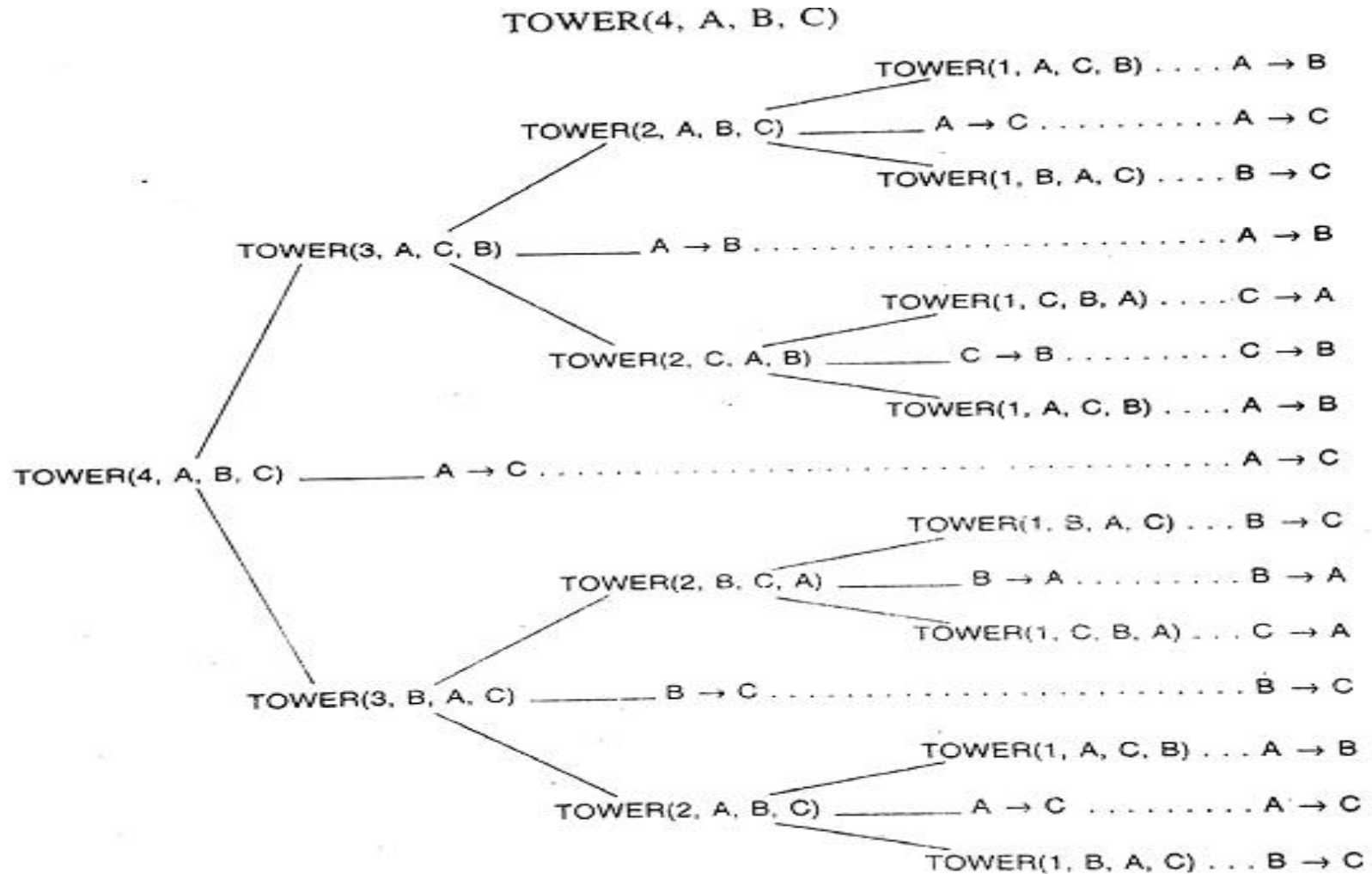


Fig. 6.17 Recursive Solution to Towers of Hanoi Problem for $n = 4$

Towers of Hanoi

Observe that the recursive solution for $n = 4$ disks consists of the following 15 moves:

A $\rightarrow$ B A $\rightarrow$ C B $\rightarrow$ C A $\rightarrow$ B C $\rightarrow$ A C $\rightarrow$ B A $\rightarrow$ B A $\rightarrow$ C
B $\rightarrow$ C B $\rightarrow$ A C $\rightarrow$ A B $\rightarrow$ C A $\rightarrow$ B A $\rightarrow$ C B $\rightarrow$ C

In general, this recursive solution requires $f(n) = 2^n - 1$ moves for n disks.

... ..

Towers of Hanoi (Algorithm)

TOWER(N, BEG, AUX, END)

This procedure gives a recursive solution to the Towers of Hanoi problem for N disks.

1. If $N = 1$, then:
 - (a) Write: $BEG \rightarrow END$.
 - (b) Return.[End of If structure.]
2. [Move $N - 1$ disks from peg BEG to peg AUX.]
Call TOWER($N - 1$, BEG, END, AUX).
3. Write: $BEG \rightarrow END$.
4. [Move $N - 1$ disks from peg AUX to peg END.]
Call TOWER($N - 1$, AUX, BEG, END).
5. Return.

6.6 QUICKSORT, AN APPLICATION OF STACKS

Let A be a list of n data items. "Sorting A " refers to the operation of rearranging the elements of A so that they are in some logical order, such as numerically ordered when A contains numerical data, or alphabetically ordered when A contains character data. The subject of sorting, including various sorting algorithms, is treated mainly in Chapter 9. This section gives only one sorting algorithm, called *quicksort*, in order to illustrate an application of stacks.

Quicksort is an algorithm of the divide-and-conquer type. That is, the problem of sorting a set is reduced to the problem of sorting two smaller sets. We illustrate this "reduction step" by means of a specific example.

Suppose A is the following list of 12 numbers:

(44) 33, 11, 55, 77, 90, 40, 60, 99, 22, 88, (66)

The reduction step of the quicksort algorithm finds the final position of one of the numbers; in this illustration, we use the first number, 44. This is accomplished as follows. Beginning with the last number, 66, scan the list from right to left, comparing each number with 44 and stopping at the first number less than 44. The number is 22. Interchange 44 and 22 to obtain the list

(22) 33, 11, 55, 77, 90, 40, 60, 99, (44), 88, 66

(Observe that the numbers 88 and 66 to the right of 44 are each greater than 44.) Beginning with 22, next scan the list in the opposite direction, from left to right, comparing each number with 44 and stopping at the first number greater than 44. The number is 55. Interchange 44 and 55 to obtain the list

22, 33, 11, (44), 77, 90, 40, 60, 99, (55), 88, 66

(Observe that the numbers 22, 33 and 11 to the left of 44 are each less than 44.) Beginning this time with 55, now scan the list in the original direction, from right to left, until meeting the first number less than 44. It is 40. Interchange 44 and 40 to obtain the list

22, 33, 11, (40), 77, 90, (44), 60, 99, 55, 88, 66

(Again, the numbers to the right of 44 are each greater than 44.) Beginning with 40, scan the list from left to right. The first number greater than 44 is 77. Interchange 44 and 77 to obtain the list

22, 33, 11, (40), (44), 90, (77), 60, 99, 55, 88, 66

(Again, the numbers to the left of 44 are each less than 44.) Beginning with 77, scan the list from right to left seeking a number less than 44. We do not meet such a number before meeting 44. This means all numbers have been scanned and compared with 44. Furthermore, all numbers less than 44 now form the sublist of numbers to the left of 44, and all numbers greater than 44 now form the sublist of numbers to the right of 44, as shown below:

22, 33, 11, 40, (44), 90, 77, 60, 99, 55, 88, 66

First sublist

Second sublist

Thus 44 is correctly placed in its final position, and the task of sorting the original list A has now been reduced to the task of sorting each of the above sublists.

The above reduction step is repeated with each sublist containing 2 or more elements. Since we can process only one sublist at a time, we must be able to keep track of some sublists for future processing. This is accomplished by using two stacks, called LOWER and UPPER, to temporarily "hold" such sublists. That is, the addresses of the first and last elements of each sublist, called its *boundary values*, are pushed onto the stacks LOWER and UPPER, respectively; and the reduction step is applied to a sublist only after its boundary values are removed from the stacks. The following example illustrates the way the stacks LOWER and UPPER are used.

Consider the above list A with $n = 12$ elements. The algorithm begins by pushing the boundary values 1 and 12 of A onto the stacks to yield

LOWER: 1 UPPER: 12

In order to apply the reduction step, the algorithm first removes the top values 1 and 12 from the stacks, leaving

LOWER: (empty) UPPER: (empty)

and then applies the reduction step to the corresponding list $A[1], A[2], \dots, A[12]$. The reduction step, as executed above, finally places the first element, 44, in $A[5]$. Accordingly, the algorithm pushes the boundary values 1 and 4 of the first sublist and the boundary values 6 and 12 of the second sublist onto the stacks to yield

LOWER: 1, 6 UPPER: 4, 12

END!!!!!!!!!!